

PREMIERE NSI

Algorithmique

Les Algorithmes Gloutons

Faire le meilleur choix à chaque étape

Niveau	Première NSI -- conforme au programme officiel BO 2019
Theme	Algorithmique -- Algorithmes gloutons
Competences visees	Comprendre le principe glouton, appliquer sur des exemples concrets, coder en Python
Prerequis	Listes Python, boucles for/while, fonctions, tris elementaires
Exemples traites	Rendu de monnaie, sac a dos, coloriage de cartes (graphes), ordonnancement

Note importante : le programme officiel de Première NSI (BO 2019) stipule que les élèves doivent être capables de "donner des exemples d'algorithmes gloutons", "comprendre que ces algorithmes ne donnent pas toujours le résultat optimal", et "résoudre le problème du rendu de monnaie".

Partie 1 - Qu'est-ce qu'un algorithme glouton ?

1.1 L'idée de base : le "bon élève gourmand"

Un algorithme glouton (en anglais : **greedy algorithm**) est un algorithme qui, à chaque étape, fait le choix qui lui semble **le meilleur du moment**, sans revenir en arrière et sans se préoccuper de la suite.

Le mot "glouton" vient de l'idée de quelqu'un qui avale tout ce qu'il peut tout de suite, sans réfléchir à long terme.

Analogie 1 : La caisse du supermarché

Imaginez que vous êtes à la caisse et que vous devez rendre 87 centimes avec le moins de pièces possibles. En France, comme dans toute la zone euro, les pièces de monnaie en centimes (souvent appelées "centimes d'euro") sont au nombre de 6.

Voici la liste exacte des pièces qui existent :

Les pièces de couleur "cuivre" (acier cuivré)

- 1 centime
- 2 centimes
- 5 centimes

Les pièces de couleur "or" (alliage appelé or nordique)

- 10 centimes
- 20 centimes
- 50 centimes

Un algorithme glouton dirait : "Je prends la pièce la plus grande qui ne dépasse pas ce qui reste à rendre."

- Il reste 87c -> je prends une pièce de 50c (reste 37c)
- Il reste 37c -> je prends une pièce de 20c (reste 17c)
- Il reste 17c -> je prends une pièce de 10c (reste 7c)
- Il reste 7c -> je prends une pièce de 5c (reste 2c)
- Il reste 2c -> je prends une pièce de 2c (reste 0c)

Résultat : 5 pièces. Était-ce le mieux possible ? Oui, dans ce cas !

1.2 Les 3 caractéristiques d'un algorithme glouton

Pour qu'un algorithme soit qualifié de "glouton", il doit respecter trois principes :

1. Choix local optimal : À chaque étape, on choisit la meilleure option disponible à cet instant.
2. Irréversibilité : On ne revient jamais sur un choix déjà fait.
3. Construction progressive : La solution se construit pièce par pièce, étape par étape.

Analogie 2 : Le randonneur perdu en montagne

Un randonneur veut atteindre le sommet le plus haut. Sa stratégie glouton : à chaque carrefour,

il choisit toujours le chemin qui monte le plus fort.

Problème : il peut arriver à un sommet local (une petite bosse) et ne jamais atteindre le vrai sommet !

-> C'est exactement la limite des algorithmes gloutons : ils ne donnent pas TOUJOURS la solution optimale globale.

1.3 Pourquoi utiliser des algorithmes gloutons ?

Avantages	Inconvénients
Simple à programmer et à comprendre Très rapide à exécuter Donne de bons résultats en pratique Suffisant pour de nombreux problèmes réels	Ne garantit pas toujours la solution optimale Il faut parfois prouver que la solution est correcte Peut rater la meilleure solution globale

Partie 2 - Exemple 1 : Le Problème du Rendu de Monnaie

2.1 Présentation du problème

C'est l'exemple officiel du programme de Première NSI. Il faut rendre une somme donnée en utilisant le **minimum de pièces**. Les pièces disponibles sont : 1c, 2c, 5c, 10c, 20c, 50c, 1 euro (100c), 2 euros (200c).

2.2 La stratégie glouton

Stratégie gloutonne pour le rendu de monnaie

Tant qu'il reste de la monnaie à rendre :

- > Choisir la pièce de plus grande valeur qui ne dépasse pas le montant restant.
- > Donner cette pièce, déduire sa valeur du montant restant.
- > Recommencer.

2.3 Exemple détaillé pas-a-pas : rendre 67 centimes

Pièces disponibles : 50c, 20c, 10c, 5c, 2c, 1c (système euro).

Etape	Reste a rendre	Plus grande piece utilisable	Reste apres	Pieces rendues jusqu'ici
1	67c	50c (≤ 67)	17c	50c
2	17c	10c (≤ 17 , car $20 > 17$)	7c	50c + 10c
3	7c	5c (≤ 7 , car $10 > 7$)	2c	50c + 10c + 5c
4	2c	2c (≤ 2)	0c -> FINI !	50c + 10c + 5c + 2c

Résultat : 4 pièces. La solution gloutonne donne ici le résultat optimal.

2.4 Le code Python - version complète et commentée

```
def rendu_monnaie_glouton(somme, pieces):
    """Rend la somme avec le minimum de pieces (algo glouton).
    somme : montant a rendre (en centimes)
    pieces : liste des pieces disponibles, trieées dans l'ordre DECROISSANT
    Retourne la liste des pieces rendues."""

    rendu = []          # La liste des pieces qu'on va rendre
    reste = somme         # Combien il reste encore a rendre

    # On parcourt les pieces de la plus grande a la plus petite
    for piece in pieces:

        # Tant que cette piece ne depasse pas ce qu'il reste
        while reste >= piece:
            rendu.append(piece)    # On ajoute cette piece
```

```

    reste = reste - piece # On deduit sa valeur

    # Si reste == 0, on a fini, inutile de continuer
    if reste == 0:
        break

    return rendu

# Pieces euro trieées par ordre decroissant
pieces_euro = [200, 100, 50, 20, 10, 5, 2, 1]

# Test : rendre 67 centimes
resultat = rendu_monnaie_glouton(67, pieces_euro)
print(resultat) # Affiche : [50, 10, 5, 2]
print(len(resultat)) # Affiche : 4 (nombre de pieces)

```

2.5 Quand l'algorithme glouton échoue : un contre-exemple !

L'algorithme glouton **ne fonctionne pas avec tous les systèmes de pièces**. Voici un exemple important :

Contre-exemple : pièces {1, 3, 4}, rendre 6

Pieces disponibles : 4, 3, 1 (ordre décroissant)

Algorithme glouton :

- > Reste 6 : prend 4 (reste 2)
- > Reste 2 : prend 1 (reste 1)
- > Reste 1 : prend 1 (reste 0)
- => Résultat glouton : $4 + 1 + 1 = 3$ pièces

Solution optimale réelle :

- => $3 + 3 = 2$ pièces (seulement 2 pièces !)

Conclusion : L'algorithme glouton a échoué. Il faudrait ici une autre approche (programmation dynamique).

Conclusion importante : Pour le système de pièces euro, l'algorithme glouton est optimal. Mais ce n'est pas toujours le cas !

Partie 3 - Exemple 2 : Le Problème du Sac à Dos (version fractionnaire)

3.1 Présentation du problème

Analogie du quotidien

Imaginez que vous partez en vacances avec une valise de 10 kg maximum. Vous avez plein d'affaires à emporter, chacune avec un poids et une utilité. Vous voulez maximiser ce que vous emportez en ne dépassant pas 10 kg. La stratégie glouton : toujours prendre en priorité ce qui a la meilleure utilité par kilo !

3.2 Les objets disponibles

Version **fractionnaire** : on peut prendre une partie d'un objet (comme de l'or en poudre). Sac : capacité max = 10 kg.

Objet	Poids (kg)	Valeur (euros)	Valeur / kg (ratio)	Rang (priorité)
Or en poudre	2	60	30 euros/kg	1er
Diamants	3	75	25 euros/kg	2eme
Argent	5	75	15 euros/kg	3eme
Bronze	4	20	5 euros/kg	4eme

3.3 Application de la stratégie gloutonne

Stratégie : prendre en priorité les objets avec le meilleur ratio valeur/poids.

Etape	Objet choisi	Quantité prise	Capacité restante	Valeur accumulée
1	Or (ratio 30)	2 kg (tout l'or)	$10 - 2 = 8$ kg	$2 \times 30 = 60$ euros
2	Diamants (ratio 25)	3 kg (tous les diamants)	$8 - 3 = 5$ kg	$60 + 75 = 135$ euros
3	Argent (ratio 15)	5 kg (tout l'argent)	$5 - 5 = 0$ kg -> PLEIN	$135 + 75 = 210$ euros

Résultat optimal : **210 euros** pour un sac de 10 kg. On a pris tout l'or, tous les diamants et tout l'argent. Le bronze n'a pas été pris car le sac était déjà plein.

3.4 Le code Python

```
def sac_a_dos_glouton(capacite, objets):
    """Resout le sac a dos fractionnel par algorithme glouton.
    capacite : poids max du sac (en kg)
    objets : liste de tuples (nom, poids, valeur)
    Retourne la valeur totale maximale."""

    # Etape 1 : Calculer le ratio valeur/poids pour chaque objet
    # et trier par ratio DECROISSANT (le meilleur en premier)
    objets_tries = sorted(objets,
                           key=lambda x: x[2]/x[1], # valeur/poids
                           reverse=True)           # ordre decroissant

    valeur_totale = 0 # Ce qu'on a accumule
    poids_restant = capacite # Ce qu'il reste dans le sac

    for nom, poids, valeur in objets_tries:
        if poids_restant <= 0: # Sac plein, on s'arrete
            break

        # Peut-on prendre tout l'objet ?
        if poids <= poids_restant:
            quantite = poids # On prend tout
        else:
            quantite = poids_restant # On prend ce qui reste de place

        valeur_totale += (quantite / poids) * valeur
        poids_restant -= quantite
        print(f"Pris {quantite}kg de {nom}")

    return valeur_totale

# Donnees : (nom, poids, valeur)
objets = [("Or", 2, 60), ("Diamants", 3, 75),
          ("Argent", 5, 75), ("Bronze", 4, 20)]

valeur = sac_a_dos_glouton(10, objets)
print(f"Valeur totale : {valeur} euros") # 210.0 euros
```

Attention : version non-fractionnaire (objets entiers)

Si on ne peut PAS couper les objets (on doit les prendre entiers ou pas du tout),

L'algorithme glouton NE DONNE PLUS la solution optimale.

Dans ce cas, il faudrait utiliser la programmation dynamique (hors programme de Première).

Au programme de Première, on étudie uniquement la version fractionnaire ou des exemples simples.

Partie 4 - Exemple 3 : Ordonnancement de Taches

4.1 Présentation du problème

Un processeur (ou une salle) doit exécuter plusieurs taches. Chaque tache a une **durée** et une **échéance** (date limite pour finir). Comment choisir l'ordre pour respecter le maximum d'échéances ?

Analogie : L'élève qui a plusieurs devoirs

Vous avez 4 devoirs à rendre cette semaine. Vous pouvez en faire un par jour.

Certains sont urgents (à rendre demain), d'autres moins (dans 3 jours).

Stratégie glouton : faites toujours en priorité le devoir le plus urgent (échéance la plus proche) !

C'est l'algorithme glouton 'Earliest Deadline First'.

4.2 Exemple pas-a-pas

Taches à faire (durée = 1 unité de temps chacune). On a 4 créneaux disponibles : t=1, t=2, t=3, t=4.

Tache	Duree (h)	Echeance (heure limite)	Priorite (earliest deadline)
A : Devoir de maths	1	t = 1	1ere (la plus urgente)
B : Rapport de francais	1	t = 2	2eme
C : Exercice de physique	1	t = 3	3eme
D : Projet d'informatique	1	t = 3	4eme (ex-aequo avec C)

Ordre glouton (Earliest Deadline First) : A -> B -> C -> D

Toutes les taches sont terminées dans les temps ! La stratégie glouton est optimale ici.

4.3 Code Python simplifié

```
def ordonnancement_glouton(taches):
    """Ordonne les taches par echeance croissante (Earliest Deadline First).
    taches : liste de tuples (nom, echeance)
    Retourne la liste des taches dans l'ordre d'execution."""

    # Trier les taches par echeance croissante (la plus urgente en premier)
    taches_triees = sorted(taches, key=lambda t: t[1])

    print("Ordre d'execution recommande :")
    for nom, echeance in taches_triees:
        print(f"  -> {nom} (echeance : t={echeance})")

    return taches_triees
```



```
# Donnees : (nom de la tache, echeance)
taches = [("Devoir maths", 1), ("Rapport francais", 2),
          ("Exercice physique", 3), ("Projet info", 3)]

ordonnancement_glouton(taches)
```

Partie 5 - Généralisation et Limites

5.1 Quand est-ce que l'algorithme glouton est optimal ?

L'algorithme glouton **donne toujours la solution optimale** dans certains cas bien spécifiques. On dit que le problème possède une **structure gloutonne**.

Problème	Algo glouton optimal ?	Remarque
Rendu de monnaie (système euro)	OUI	Prouvable mathématiquement
Rendu de monnaie (système quelconque)	PAS TOUJOURS	Voir contre-exemple partie 2
Sac à dos fractionnel	OUI	Si on peut prendre des fractions d'objets
Sac à dos entier (0/1)	NON	Nécessite programmation dynamique
Ordonnancement EDF	OUI	Valable si toutes les tâches ont durée = 1
Coloriage de carte minimale	APPROCHE SEULEMENT	Donne une bonne solution, pas forcément la meilleure

5.2 Vocabulaire important à retenir

Terme	Définition
Algorithme glouton	Algorithme qui fait, à chaque étape, le meilleur choix local sans jamais revenir en arrière.
Choix local optimal	La meilleure décision possible au moment où on la prend, en ignorant les étapes suivantes.
Solution optimale	La meilleure solution possible parmi toutes les solutions existantes (pas seulement une bonne solution).
Heuristique	Méthode qui donne une bonne solution rapidement, sans garantir que c'est la meilleure. L'algo glouton est une heuristique.
Complexité	Mesure du nombre d'opérations. Les algos gloutons ont souvent une très bonne complexité (rapides).

Partie 6 - Exercices d'Application

Exercice 1 - Rendu de monnaie (calcul manuel)

Enoncé

On souhaite rendre 93 centimes avec le système de pièces euro :

pièces = [200, 100, 50, 20, 10, 5, 2, 1] centimes

1. Appliquez l'algorithme glouton étape par étape. Complétez le tableau suivant.
2. Combien de pièces avez-vous utilisées au total ?
3. Quelle serait la valeur retournée par `rendu_monnaie_glouton(93, pieces_euro)` ?

Etape	Reste a rendre	Plus grande piece utilisable	Reste apres	Pieces rendues
1	93c			
2				
3				
4				
5				

Exercice 2 - Sac a dos glouton

Enoncé

Un randonneur peut porter 8 kg. Il veut maximiser la valeur de ce qu'il emporte.

Objets disponibles :

- Boite de conserves : 4 kg, valeur = 20 points
- Lampe de poche : 1 kg, valeur = 15 points
- Tente : 6 kg, valeur = 30 points
- Trousse premiers secours : 2 kg, valeur = 22 points

1. Calculez le ratio valeur/poids pour chaque objet.
2. Classez les objets par ratio décroissant.
3. Appliquez l'algorithme glouton (version fractionnaire : on peut couper les objets).
4. Quelle est la valeur totale emportée ?

Objet	Poids (kg)	Valeur	Ratio val/poids	Rang
Boite de conserves	4	20		
Lampe de poche	1	15		
Tente	6	30		
Trousse 1ers secours	2	22		

Exercice 3 - Question de cours (type contrôle)

Questions

1. Donnez la définition d'un algorithme glouton en utilisant vos propres mots.
2. Citez les 3 caractéristiques d'un algorithme glouton.
3. L'algorithme glouton garantit-il toujours la solution optimale ? Justifiez avec un exemple.
4. Quelle est la stratégie glouton pour le problème du rendu de monnaie ?
5. On veut rendre 15 centimes avec les pièces {1, 6, 9}. L'algo glouton donne-t-il la solution optimale ? Montrez pourquoi.

Exercice 4 - Codage Python

Enonce

Ecrivez une fonction Python `rendu_monnaie(somme)` qui :

- prend en parametre un montant en centimes (entier positif)
- utilise les pieces euro : [200, 100, 50, 20, 10, 5, 2, 1]
- renvoie un dictionnaire indiquant le nombre de pieces de chaque type

Exemple : `rendu_monnaie(93)`

-> `{'50': 1, '20': 2, '2': 1, '1': 1}`
(50 + 20 + 20 + 2 + 1 = 93 centimes)

Conseil : commencez par ecrire la version qui renvoie la liste simple, puis adaptez pour renvoyer le dictionnaire.

Partie 7 - Corrige des Exercices

Corrige - Exercice 1

1. Trace de l'algorithme glouton pour 93 centimes :

Etape	Reste a rendre	Plus grande piece utilisable	Reste apres	Pieces rendues
1	93c	50c (≤ 93)	43c	50c
2	43c	20c (≤ 43 , car $50 > 43$)	23c	50+20c
3	23c	20c (≤ 23)	3c	50+20+20c
4	3c	2c (≤ 3 , car $5 > 3$)	1c	50+20+20+2c
5	1c	1c (≤ 1)	0c -> FINI	50+20+20+2+1c

2. Total : **5 pieces** (une de 50c, deux de 20c, une de 2c, une de 1c).

3. rendu_monnaie_glouton(93, pieces_euro) retourne : **[50, 20, 20, 2, 1]**

Corrige - Exercice 2

Objet	Poids (kg)	Valeur	Ratio val/poids	Rang
Lampe de poche	1	15	15 pts/kg	1er
Trousse 1ers secours	2	22	11 pts/kg	2eme
Boite de conserves	4	20	5 pts/kg	3eme
Tente	6	30	5 pts/kg	4eme

Application glouton (capacite 8 kg) :

- Lampe (1 kg) : prise entiere. Reste 7 kg. Valeur = 15.
- Trousse (2 kg) : prise entiere. Reste 5 kg. Valeur = 15 + 22 = 37.
- Conserves (4 kg) : prise entiere. Reste 1 kg. Valeur = 37 + 20 = 57.
- Tente (6 kg, il reste 1 kg) : on prend 1/6 de la tente. Valeur += $1/6 \times 30 = 5$.

Valeur totale : **57 + 5 = 62 points** pour 8 kg.

Corrige - Exercice 3

- Un algorithme glouton est un algorithme qui, a chaque etape, choisit la meilleure option disponible a cet instant, sans jamais revenir sur ses choix precedents.
- Les 3 caracteristiques : (a) Choix local optimal a chaque etape, (b) Irreversibilite des choix, (c) Construction progressive de la solution.
- Non, l'algorithme glouton ne garantit pas toujours l'optimum. Contre-exemple : pieces {1, 3, 4}, rendre 6. L'algo glouton donne 4+1+1 (3 pieces) alors que 3+3 (2 pieces) est meilleur.

4. Stratégie glouton pour le rendu de monnaie : à chaque étape, choisir la pièce de plus grande valeur qui ne dépasse pas le montant restant à rendre.
5. Pièces {1, 6, 9}, rendre 15. Glouton : $9 + 6 = 15$ (2 pièces). Solution optimale : $9 + 6 = 15$ (2 pièces). Ici l'algo glouton donne bien la solution optimale. Mais si on demandait 12 : glouton donne $9+1+1+1$ (4 pièces) alors que $6+6$ (2 pièces) est mieux.

Corrige - Exercice 4

```
def rendu_monnaie(somme):
    """Rend la somme en centimes avec le moins de pieces possible (systeme euro).
    Retourne un dictionnaire {valeur_piece: nombre_de_pieces}."""

    pieces_euro = [200, 100, 50, 20, 10, 5, 2, 1]
    resultat = {}      # Dictionnaire vide
    reste = somme

    for piece in pieces_euro:
        if reste >= piece:
            # Cette piece est utilisable
            nb = reste // piece      # Combien de fois cette piece ?
            resultat[str(piece)] = nb # On stocke dans le dico
            reste = reste % piece    # Ce qu'il reste apres

        if reste == 0:
            # Plus rien a rendre
            break

    return resultat

# Test
print(rendu_monnaie(93))
# -> {'50': 1, '20': 2, '2': 1, '1': 1}

print(rendu_monnaie(87))
# -> {'50': 1, '20': 1, '10': 1, '5': 1, '2': 1}
```

Fiche Recapitulative - L'Essentiel a Retenir

LES POINTS CLES

Definition

Un algorithme glouton fait, a chaque etape, le meilleur choix LOCAL sans jamais revenir en arriere.

Les 3 proprietes

1. Choix local optimal : toujours la meilleure option disponible maintenant.
2. Irreversibilite : on ne revient jamais sur un choix.
3. Construction progressive : la solution se batit etape par etape.

Exemples cles au programme

Rendu de monnaie : toujours prendre la plus grande piece qui ne depasse pas le reste.

Sac a dos fractionnel : trier par ratio valeur/poids decroissant, prendre dans l'ordre.

Ordonnancement : traiter d'abord la tache a l'echeance la plus proche (EDF).

Attention !

L'algo glouton ne donne PAS toujours la solution optimale.

Il faut verifier : pieces {1,3,4}, rendre 6 -> glouton donne 4+1+1 (3 pieces) mais 3+3 (2 pieces) est meilleur.

Quand l'optimum n'est pas garanti, l'algo glouton reste une heuristique (bonne approche, pas forcément la meilleure).

Squelette Python a connaitre

```
for piece in sorted(disponibles, reverse=True):    # Trier par valeur
    decroissante
    while reste >= piece:                          # Tant qu'on peut
        utiliser la piece
        prendre(piece); reste -= piece            # Prendre et deduire
```

Conseils pour le controle

1. Pour une trace d'execution : faites TOUJOURS un tableau etape par etape.
2. Verifiez que vous triez dans le bon sens (decroissant pour monnaie et sac a dos).
3. Si on vous demande si l'algo est optimal : donnez la reponse ET justifiez avec un exemple.
4. Pour le code Python : n'oubliez pas de trier la liste AVANT la boucle principale.
5. Le mot 'glouton' vient de : faire le meilleur choix maintenant, sans penser a demain.